0778. Swim in Rising Water

| Category: Advanced Graphs | LeetCode: https://leetcode.com/problems/swim-i

Problem

- N x n grid of elevations. At time t you can swim between adjacent cells with elevation <= t. Return the minimum t to get from (0,0) to (n-1,n-1).

Intuition

- The cost of a path is its maximum cell, and we want the path minimising that maximum (a "minimax" path).
- Dijkstra with "distance = max elevation so far" works because the max never decreases along a path.

Brute Force

- Binary search on t plus a BFS feasibility check: O(n^2 log n^2) - also a good alternative.

Optimal Approach

- Min-heap of (time, r, c) starting with (grid[0][0], 0, 0).
- Pop the smallest; if it is the target, return time.
- Push unvisited neighbours with max(time, grid[nr][nc]).

Worked Example

```
[[0,2],[1,3]].

pop (0,0,0)  push (2,0,1) (1,1,0)
pop (1,1,0)  push (3,1,1)
pop (2,0,1)  push (3,1,1) again
pop (3,1,1)  target -> 3
```

Java Solution

```java
public class SwimInRisingWater {
    public int swimInWater(int[][] grid) {
        int n = grid.length;
        boolean[][] seen = new boolean[n][n];
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));
        pq.add(new int[]{grid[0][0], 0, 0});
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        while (!pq.isEmpty()) {
            int[] cur = pq.poll();
            int t = cur[0], r = cur[1], c = cur[2];
            if (seen[r][c]) continue;
            seen[r][c] = true;
            if (r == n - 1 && c == n - 1) return t;
            for (int[] d : dirs) {
                int nr = r + d[0], nc = c + d[1];
                if (nr >= 0 && nc >= 0 && nr < n && nc < n && !seen[nr][nc])
                    pq.add(new int[]{Math.max(t, grid[nr][nc]), nr, nc});
            }
        }
        return -1;
    }
}
```

Complexity

- Time: O(n^2 log n). Space: O(n^2).

Pitfalls

- Mark visited when popping (or when pushing with care) to avoid reprocessing.